

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное
учреждение высшего образования

«Ярославский государственный университет им. П.Г.Демидова»

Кафедра компьютерной безопасности и
математических методов обработки информации

Курсовая работа

Методы комбинаторной теории групп в современной криптографии

Научный руководитель
профессор, д.-р. физ.-мат. наук
_____ В. Г. Дурнев
« ____ » _____ 2026 г.

Студент группы КБ-41СО
_____ Т. А. Хаирнуров
« ____ » _____ 2026 г.

Ярославль 2026

Содержание

Введение	3
1. Предварительные сведения	4
2. Протокол Стикеля	7
2.1. Проблема разложения	7
2.2. Протокол обмена ключами	8
2.3. Платформа протокола	10
3. Реализация протокола	11
3.1. Архитектура разложения	11
3.2. Ключевые алгоритмы	12
3.2.1. Умножение циклических многочленов	12
3.2.2. Быстрое возведение матрицы в степень	12
3.2.3. Основной протокол	13
3.3. Пользовательский интерфейс	14
3.4. Зависимости	15
4. Заключение	15
Список литературы	16
Приложение А	16

Введение

В начале 2000-х годов активно начала развиваться некоммутативная криптография. Её основная идея — перенести криптографические конструкции в некоммутативные группы, где вместо проблемы дискретного логарифмирования центральное место занимают проблема поиска сопряжённого элемента (Conjugacy Search Problem, CSP) и проблема разложения (Decomposition Search Problem). Эти задачи, как правило, не имеют эффективных решений в общем случае и поэтому представляют интерес с криптографической точки зрения.

Одним из первых протоколов некоммутативного обмена ключами стал протокол Стикеля. Он представляет собой прямую аналогию классического протокола Диффи-Хеллмана, адаптированную к некоммутативным структурам. Несмотря на то, что оригинальная реализация протокола на матричных группах подвергалась линейно-алгебраическим атакам, он остается ценным исследовательским инструментом, который позволяет наглядно продемонстрировать:

- переход от абелевых к некоммутативным структурам;
- влияние выбора платформы на безопасность;

В настоящей курсовой работе будет реализована версия протокола Стикеля на полугруппе матриц над кольцом циклических многочленов $\mathbb{Z}_n[x]/(x^{N+1} - 1)$.

1. Предварительные сведения

Определение 1. Множество S с бинарной операцией $*$ называется *полугруппой*, если:

$$\bullet (\forall a, b, c \in S) : (a * b) * c = a * (b * c)$$

Определение 2. Множество G с бинарной операцией $\cdot$ называется *группой*, если:

1. $(\forall a, b, c \in G) : (a \cdot b) \cdot c = a \cdot (b \cdot c)$
2. $(\exists e \in G)(\forall a \in G) : a \cdot e = e \cdot a = a$
3. $(\forall a \in G)(\exists a^{-1} \in G) : a \cdot a^{-1} = a^{-1} \cdot a = e$

Если групповая операция коммутативна, то есть $(\forall a, b \in G)(a \cdot b = b \cdot a)$, то группа называется *коммутативной* или *абелевой*. Кроме того, если $|G| < \infty$, то группа G называется *конечной*.

Пример 1. Кольцо целых чисел $\mathbb{Z}$ с операцией сложения является абелевой группой.

Пример 2. Множество $GL_n(F)$ невырожденных квадратных матриц размера $n \times n$ над полем F с операцией обычного матричного умножения является группой.

Определение 3. Подмножество H группы G называется *подгруппой* группы G (обозначается $H \leq G$), если:

1. $(\forall a, b \in H) : a \cdot b \in H$
2. $e \in H$
3. $(\forall a \in H) : a^{-1} \in H$

Пример 3. $\mathbb{Z} \subset \mathbb{R}$ является подгруппой относительно сложения.

Определение 4. Если $N \leq G$ и

$$(\forall g \in G)(\forall n \in N) : g \cdot n \cdot g^{-1} \in N,$$

то N называется *нормальной подгруппой* группы G (обозначается $N \trianglelefteq G$).

Определение 5. *Порядком элемента g группы G* называется наименьшее $n \in \mathbb{N}$ со свойством $g^n = e$, если такие n существуют, и бесконечность — в противном случае. Порядок g обозначается $\text{ord}(g)$

Определение 6. Множество $Z(x) = \{g \in G : gx = xg\}$ называется *централизатором* элемента x .

Утверждение 1. Централизатор $Z(x)$ элемента $x \in G$ является подгруппой.

▷

1. Пусть $a, b \in Z(x) : (a \cdot b) \cdot x = a \cdot b \cdot x = a \cdot x \cdot b = x \cdot (a \cdot b)$.

2. $e \cdot x = x \cdot e$.

3. Пусть $a \in Z(x) :$

$$a^{-1} \cdot x = a^{-1} \cdot x \cdot a \cdot a^{-1} = a^{-1} \cdot a \cdot x \cdot a^{-1} = x \cdot a^{-1},$$

следовательно $a^{-1} \in Z(x)$.

◀

Определение 7. Множество $Z(G) = \{z \in G : (\forall g \in G)(zg = gz)\}$ называется *центром* группы G .

Утверждение 2. Центр $Z(G)$ группы G является подгруппой.

Определение 8. Отображение $\varphi : (G, \cdot) \rightarrow (G', \circ)$ называется *гомоморфизмом* группы G в группу G' , если:

$$\varphi(a \cdot b) = \varphi(a) \circ \varphi(b),$$

для любых $a, b \in G$. Если φ биективно, то φ называется *изоморфизмом*.

Теорема 1 (Первая теорема о гомоморфизме). Пусть $f : G \rightarrow H$ — гомоморфизм групп. Тогда

$$\text{im}(f) \simeq G/\ker(f)$$

Пусть G группа, $A \subseteq G$ произвольное подмножество. Через $\langle A \rangle$ будем обозначать подгруппу группы G порожденную A (пересечение всех подгрупп G содержащих A). Нетрудно убедиться, что

$$\langle A \rangle = \{a_{i_1}^{\varepsilon_1} \dots a_{i_n}^{\varepsilon_n} \mid a_{i_j} \in A, \varepsilon_j \in \{1, -1\}, n \in \mathbb{N}\}$$

Пусть X произвольное множество. Словом w в алфавите X будем называть конечную (возможно пустую) последовательность элементов X

$$w = x_1 \dots x_n, \quad x_i \in X$$

Число n называется *длиной* слова w (обозначается $|w|$). *Пустое слово* обозначается ε и $|\varepsilon| = 0$. Пусть $X^{-1} = \{x^{-1} \mid x \in X\}$ — алфавит «обратных» букв. Объединение $X^{\pm 1} = X \cup X^{-1}$ будем называть *групповым алфавитом*.

Выражение вида

$$w = x_{i_1}^{\varepsilon_1} \dots x_{i_n}^{\varepsilon_n},$$

где $x_{i_j} \in X^{\pm 1}$, $\varepsilon_j \in \{1, -1\}$ будем называть *групповым словом* в X .

Групповое слово w называется *редуцированным*, если для всех i $y_i \neq y_{i+1}^{-1}$, то есть w не содержит подслово вида yy^{-1} для любого символа $y \in X^{\pm 1}$. Пустое слово будем считать редуцированным.

Если $X \subseteq G$, тогда любое групповое слово $w = x_{i_1}^{\varepsilon_1} \dots x_{i_n}^{\varepsilon_n}$ в X определяет уникальный элемент в G , равный произведению $x_{i_1}^{\varepsilon_1} \dots x_{i_n}^{\varepsilon_n}$ элементов $x_{i_j}^{\varepsilon_j} \in G$. Будем считать, что пустое слово ε определяет нейтральный элемент $e \in G$.

Определение 9. Группа G называется *свободной группой*, если существует такое порождающее множество $X \subseteq G$, что любое непустое редуцированное групповое слово в X определяет нетривиальный элемент G

Тогда X называется *свободным базисом*, а G называется *свободной группой над X* (обозначается $F(X)$).

Из этого определения следует, что любой элемент $F(X)$ может быть определен редуцированным групповым словом в X . Более того, разные редуцированные групповые слова в X определяют разные элементы G .

Свободные группы обладают следующим универсальным свойством

Теорема 2 (Универсальное свойство). Пусть G группа с порождающим множеством $X \subseteq G$. Тогда G свободная группа над X тогда и только тогда, когда справедливо универсальное свойство:

любое отображение $\varphi : X \rightarrow H$ из X в группу H продолжается единственным образом до гомоморфизма

$$\varphi^* : G \longrightarrow H$$

так, что следующая диаграмма коммутативна

$$\begin{array}{ccc} X & \xrightarrow{i} & G \\ & \searrow \varphi & \downarrow \varphi^* \\ & & H \end{array}$$

Универсальное свойство свободных групп позволяет описывать произвольные группы в терминах *образующих* и *определяющих соотношений*.

Пусть G – группа с порождающим множеством X . Из универсального свойства следует, что существует гомоморфизм $\psi : F(X) \rightarrow G$ такой, что $\psi(x) = x$ для $x \in X$. Следовательно, ψ сюръективно, а значит по Теореме 1

$$G \simeq F(X)/\ker(\psi)$$

В этом случае $\ker(\psi)$ рассматривается как множество соотношений в G , а групповое слово $w \in \ker(\psi)$ называется *соотношением* группы G в образующих X . Если подмножество $R \subseteq \ker(\psi)$ порождает $\ker(\psi)$ как нормальную подгруппу в $F(X)$, то оно называется множеством *определяющих соотношений* группы G относительно X . Пара $\langle X \mid R \rangle$ называется *заданием* группы G ; оно определяет G однозначно с точностью до изоморфизма. Задание $\langle X \mid R \rangle$ называется *конечным*, если оба множества X и R конечны. Группа называется *конечно представленной*, если она имеет хотя бы одно конечное задание. Задания обеспечивают универсальный способ описания групп. В частности, конечно представленные группы допускают конечные описания, например

$$G = \langle x_1, x_2, \dots, x_n \mid r_1, r_2, \dots, r_n \rangle$$

2. Протокол Стикеля

2.1. Проблема разложения

Прежде, чем перейти к протоколу Стикеля, нужно сформулировать несколько связанных алгоритмических задач (для групп).

Задача 1 (Conjugacy search problem, CSP). Дано рекурсивное задание группы G и два сопряженных элемента $g, h \in G$. Необходимо найти элемент $x \in G$ такой, что $x^{-1}gx = h$.

Задача 2 (Decomposition search problem, DSP). Дано рекурсивное задание группы G , две рекурсивно порожденные подгруппы $A, B \leq G$ и два элемента $g, h \in G$. Необходимо найти два элемента $x \in A$ и $y \in B$ такие, что

$$x \cdot g \cdot y = h$$

при условии, что хотя бы одна такая пара существует.

Отметим, что некоторые x и y , удовлетворяющие равенству $x \cdot g \cdot y = h$ существуют всегда (например, если $x = 1, y = g^{-1}h$), поэтому смысл задачи состоит в том, чтобы они удовлетворяли условиям $x \in A, y \in B$.

Утверждение 3. Если CSP в группе G разрешима, тогда и DSP разрешима для коммутирующих подгрупп $A, B \leq G$ (т.е. $a \cdot b = b \cdot a$ для всех $a \in A, b \in B$).

▷ Пусть $h = xgy$, где $g, h \in G, x \in A, y \in B$. Мы хотим найти x и y . Выберем произвольный $a_1 \in A$. Тогда

$$\begin{aligned} [h, a_1] &= [xgy, a_1] = xgya_1y^{-1}g^{-1}x^{-1}a_1^{-1} = xga_1g^{-1}x^{-1}a_1^{-1} = \\ &= (a_1)^{x^{-1}g^{-1}}a_1^{-1} = \left((a_1)^{g^{-1}}\right)^{x^{-1}}a_1^{-1} = h', \end{aligned}$$

где запись $[a, b] = aba^{-1}b^{-1}$ — коммутатор элементов a и b , а $a^b = b^{-1}ab$ — сопряжение.

Так как a_1 известен, то мы можем домножить результат на a_1 справа. Получим

$$h'' = h'a_1 = \left((a_1)^{g^{-1}}\right)^{x^{-1}}$$

Теперь нужно найти $x \in A$ при известных $h'', a_1, (a_1)^{g^{-1}}$, то есть мы свели проблему к поиску сопрягающего элемента. Аналогично, можно восстановить $y \in B$.



2.2. Протокол обмена ключами

Протокол обмана ключами Стиклея напоминает классический протокол Диффи-Хеллмана.

Пусть G публичная неабелева конечная (полу)группа, $a, b \in G$ открытые элементы такие, что $ab \neq ba$. Протокол заключается в следующем. Пусть N и M порядки a и b соответственно.

1. Алиса выбирает 2 случайных натуральных числа $n < N, m < M$ и отправляет $u = a^n b^m$ Бобу.
2. Боб выбирает 2 случайных натуральных числа $r < N, s < M$ и отправляет $v = a^r b^s$ Алисе.

3. Алиса вычисляет $K_A = a^n v b^m = a^{n+r} b^{m+s}$.
4. Боб вычисляет $K_B = a^r u b^s = a^{r+n} b^{s+m}$.

Таким образом, Алиса и Боб получают один и тот же элемент $K = K_A = K_B$, который можно использовать как общий ключ.

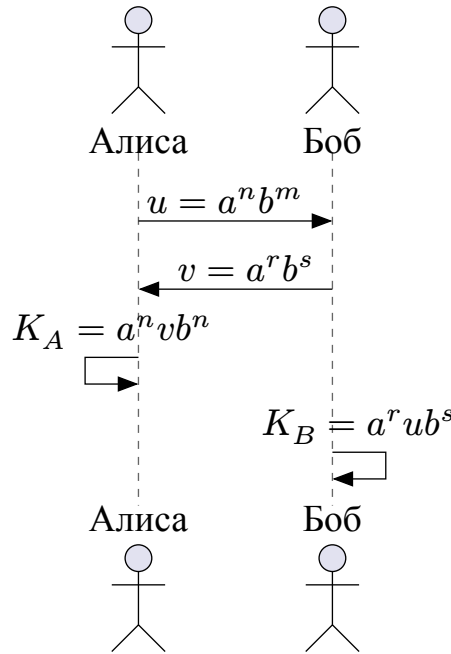


Рис. 1. Протокол Стиклея

Также, существует более общая версия этого протокола.

Пусть $w \in G$ открытый элемент.

1. Алиса выбирает 2 случайных натуральных числа $n < N, m < M$, элемент $c_1 \in Z(G)$ и отправляет Бобу $u = c_1 a^n w b^m$.
2. Боб выбирает 2 случайных натуральных числа $r < N, s < M$, элемент $c_2 \in Z(G)$ и отправляет Алисе $v = c_2 a^r w b^s$.
3. Алиса вычисляет $K_A = c_1 a^n v b^m = c_1 c_2 a^{n+r} w b^{m+s}$.
4. Боб вычисляет $K_B = c_2 a^r u b^s = c_1 c_2 a^{n+r} w b^{m+s}$.

Таким образом, Алиса и Боб получают один и тот же элемент $K = K_A = K_B$.

Подчеркнем, что для работы протокола G необязательно должно быть группой, хватит и случая, где G — полугруппа (что, на самом деле, даже лучше).

Теперь стоит обсудить общий подход (т.е. независящий от выбора G) анализа безопасности протокола. Первое наблюдение заключается в том, что чтобы получить общий секрет K , противнику (Еве) достаточно найти любые элементы $x, y \in G$ такие, что $xa = ax, yb = by, u = xwy$. Действительно, имея

такие x, y , Ева может использовать передачу Боба $v = c_2 a^r w b^s$ чтобы вычислить

$$xvy = xc_2 a^r w b^s y = c_2 a^r x w y b^s = c_2 a^r u b^s = K.$$

Отсюда следует, что умножение на c_i не усиливает стойкость протокола. Более того, Еве даже не нужно восстанавливать экспоненты n, m, r, s ; вместо этого она может решить систему уравнений $xa = ax, yb = by, u = xwy$, где a, b, w, u – известны, и x, y – неизвестные элементы G . Решение этой системы есть ничто иное как решение Задачи 2.

2.3. Платформа протокола

Традиционно, для групп, выбираемых в качестве платформы, выдвигаются следующие требования:

1. Группа должна быть хорошо изучена.
2. Проблема слова в G должна иметь быстрое (линейное или квадратичное) решение детерминированным алгоритмом. Более того, должна существовать эффективно вычислимая «нормальная форма» для элементов G .
3. Должен существовать способ маскировки элементов G так, чтобы было невозможно восстановить, скажем x и y из произведения xu простым осмотром.
4. G должна быть группой сверхполиномиального (то есть экспоненциального или «промежуточного») роста. Это означает, что количество элементов длины n в G должно расти быстрее любого полинома от n ; Это необходимо для предотвращения атак полным перебором.

В качестве платформы для протоколов некоммутативной криптографии естественно рассматривать (полу)группы матриц над конечными коммутативными кольцами, потому что умножение матриц некоммутативно, но элементы матриц, взятые из коммутативного кольца, обеспечивают хороший механизм сокрытия.

Также, у матриц нет проблемы с «нормальной формой» (неформально, нормальная форма – это способ представления элементов группы в виде простых выражений, который обеспечивает однозначность и упрощает работу с ними), поскольку (полу)группы матриц допускают иное описание, так что способ представления их элементов в виде квадратных таблиц фактически является «естественной» нормальной формой.

Причина, по которой стоит брать конечное кольцо, заключается в том, что это хорошо для диффузии. Конечные кольца R являются периодическими, что означает, что для любого $u \in R$ существуют положительные целые числа m, k такие, что $u^m = u^k$. Периодичность хороша для диффузии, поскольку порождает динамическую систему, а динамические системы с большим числом состояний обычно демонстрируют очень сложное поведение (например, знаменитая « $3x + 1$ » задача).

В качестве платформы для протокола Стикеля я выбрал полугруппу матриц над кольцом циклических многочленов над $\mathbb{Z}_n$. Циклические многочлены — это выражения вида $\sum_{k=0}^N a_k x^k$ с обычным сложением и умножением по правилу $x^i \cdot x^j = x^{i+j \bmod (N+1)}$, то есть это фактор-кольцо $\mathbb{Z}_n[x]/(x^{N+1} - 1)$. Идеал порожденный многочленом $x^{N+1} - 1$ довольно прост, вычисления в этом фактор-кольце довольно эффективны. Кроме того, большое ключевое пространство обеспечивается с низкими затратами; например, при $n = 100, N = 20$ существует $100^{21} = 10^{42}$ циклических многочленов, что дает 10^{168} матриц размера 2×2 над этим кольцом.

3. Реализация протокола

В этом разделе описывается разработанное приложение, реализующее протокол Стикеля на полугруппе матриц над кольцом циклических многочленов $\mathbb{Z}_n[x]/(x^{N+1} - 1)$. Код написан на языке Python и использует библиотеку Streamlit для построения веб-интерфейса. Полные исходные тексты приведены в приложении. Файлы организованы в следующей структуре:

```
project/
├─ app.py
├─ CyclicPolynomial.py
├─ protocol.py
├─ utils.py
└─ requirements.txt
```

3.1. Архитектура разложения

Программа написана на языке Python с использованием библиотеки Streamlit для веб-интерфейса. Логика разделена на модули:

- **CyclicPolynomial**: определяет класс `CyclicPolynomial`, который представляет циклический многочлен. Хранит коэффициенты по модулю n , степень N и реализует основные арифметические операции с приведением по модулю, а также возведение в степень.

- `utils.py`: содержит вспомогательные функции: создание единичной матрицы из полиномов, быстрое возведение матрицы в степень и проверку, является ли матрица единичной.
- `protocol.py`: реализует класс `StickelProtocol`, который выполняет шаги протокола (генерация секретных чисел, вычисление $u = A^n B^m$ и $v = A^r B^s$, формирование общего ключа).
- `app.py`: главный файл с интерфейсом Streamlit. Позволяет задать параметры n (модуль кольца), N (максимальную степень), размер матриц, плотность ненулевых коэффициентов. Запускает протокол и выводит матрицы в формате LaTeX, а также результат совпадения ключей.

Все матрицы хранятся в виде массивов NumPy с типом элемента `object`, что позволяет хранить в ячейках экземпляры `CyclicPolynomial`.

3.2. Ключевые алгоритмы

3.2.1. Умножение циклических многочленов

Умножение выполняется по правилу $x^i x^j = x^{i+j \bmod (N+1)}$ с последующим приведением коэффициентов по модулю n . Листинг 1 показывает этот метод:

```
def _mul_poly(self, other):
    """Умножение многочленов"""
    res = CyclicPolynomial(self.N, self.modulus)
    for i in range(self.M):
        for j in range(self.M):
            k = (i + j) % self.M
            res[k] += self[i] * other[j]

    return res
```

Листинг 1. Умножение циклических многочленов

3.2.2. Быстрое возведение матрицы в степень

Для вычисления A^n используется бинарный алгоритм, реализованный в функции `mat_row` (модуль `utils.py`). Листинг 2 демонстрирует этот алгоритм:

```

def mat_pow(mat, exponent):
    """Возведение матрицы (np.ndarray, dtype=object) в целую
    неотрицательную степень"""
    size = mat.shape[0]

    # Достаем параметры кольца из первого элемента матрицы
    sample_poly = mat[0, 0]
    N = sample_poly.N
    modulus = sample_poly.modulus

    result = get_identity_poly_matrix(size, N, modulus)
    base = mat
    e = exponent
    while e > 0:
        if e & 1:
            result = result @ base
            base = base @ base
            e >>= 1
    return result

```

Листинг 2. Бинарное возведение матрицы в степень

3.2.3. Основной протокол

Класс StickelProtocol инкапсулирует логику протокола. Метод run генерирует секретные числа для Алисы и Боба, обменивается сообщениями и вычисляет ключи:

```

def run(self):
    """Запускает полный протокол и возвращает ключи, а также
    промежуточные значения."""
    u, n, m = self.alice_step()
    v, r, s = self.bob_step()
    K_alice = self.alice_key(v, n, m)
    K_bob = self.bob_key(u, r, s)
    return {
        "u": u,
        "v": v,
        "n": n,
        "m": m,
        "r": r,
        "s": s,
        "K_alice": K_alice,
        "K_bob": K_bob,
        "keys_match": np.array_equal(K_alice, K_bob),
    }

```

Листинг 3. Метод run

Полный код доступен в приложении.

3.3. Пользовательский интерфейс

Интерфейс реализован с помощью библиотеки Streamlit. После запуска команды `streamlit run app.py` пользователь видит боковую панель для ввода параметров (модуль n , максимальная степень N , размер матриц, плотность). По нажатию кнопки генерируются матрицы A и B , причем проверяется условие $AB \neq BA$. Затем выполняются шаги протокола и результаты отображаются в формате LaTeX.

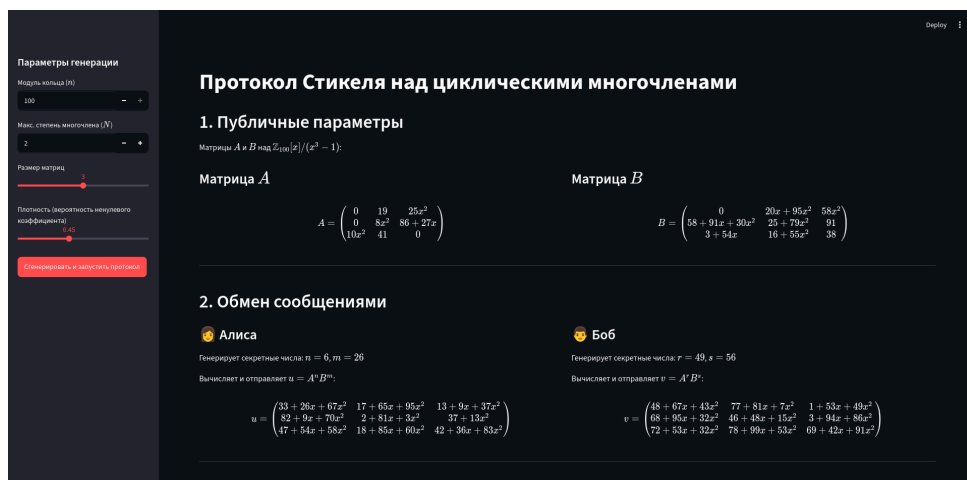


Рис. 2. Скриншот веб-интерфейса приложения

3.4. Зависимости

Для воспроизведения окружения необходимо установить библиотеки, перечисленные в файле `requirements.txt`, который приведен в листинге 4.

```
streamlit>=1.28.0  
numpy>=1.24.0
```

Листинг 4. Файл `requirements.txt`

Зависимости устанавливаются командой `pip install -r requirements.txt`.

4. Заключение

В ходе выполнения данной курсовой работы были изучены основные понятия комбинаторной теории групп, необходимые для понимания некоммутативных криптографических протоколов: группы, подгруппы, гомоморфизмы, свободные группы, задание групп образующими и определяющими соотношениями. Особое внимание уделено алгоритмическим проблемам, лежащим в основе некоммутативной криптографии — проблеме поиска сопряженного элемента (CSP) и проблеме разложения (DSP), к которой сводится криптоанализ протокола Стикеля.

Был подробно разобран протокол обмена ключами Стикеля, являющийся некоммутативным аналогом классического протокола Диффи-Хеллмана. Рассмотрены требования к платформе протокола и обоснован выбор подгруппы матриц над кольцом циклических многочленов $\mathbb{Z}_n[x]/(x^{N+1} - 1)$ как компромисса между эффективностью вычислений, стойкостью к некоторым атакам и простотой реализации.

В практической части разработано приложение на языке Python с веб-интерфейсом на базе Streamlit, реализующее полный цикл протокола: генерацию публичных матриц A и B с заданными параметрами (n , N , размер, плотность), формирование секретных чисел Алисой и Бобом, вычисление промежуточных значений $u = A^n B^m$ и $v = A^r B^s$, а также итоговых ключей K_A и K_B . Корректность работы протокола подтверждается совпадением полученных ключей.

Разработанное программное обеспечение может служить как демонстрационный стенд для изучения некоммутативной криптографии, так и основа для дальнейших исследований — например, для анализа стойкости протокола при различных параметрах кольца или для адаптации к другим алгебраическим структурам.

Список литературы

1. Магнус В., Каррас А., Солитэр Д. Комбинаторная теория групп. 1974. 456 с.
2. Каргаполов М.И., Мерзляков Ю.И. Основы теории групп. 1982. 288 с.
3. Дурнев В.Г., Зеткина О.В. Методы комбинаторной теории групп в современной криптографии. 2017. 52 с.
4. Винберг Э.Б. Курс алгебры. 2024. 592 с.
5. Myasnikov A., Shpilrain V., Ushakov A. Non-commutative Cryptography and Complexity of Group-theoretic Problems. 2011. 385 с.

Приложение А

Полный коды всех модулей приведены ниже.

Модуль CyclicPolynomial.py

```
class CyclicPolynomial:
    """Многочлен степени <= N с умножением  $x^i * x^j = x^{i+j}$ 
    (mod (N + 1)) над  $\mathbb{Z}\mathbb{Z}_n$ """

    def __init__(self, N, modulus, coeffs=None):
        """
        :param N: максимальная степень (порядок циклической группы
        мономов)
        :param modulus: n - модуль кольца  $\mathbb{Z}\mathbb{Z}_n$ 
        :param coeffs: список, кортеж или словарь {степень:
        коэффициент}
        """
        if modulus <= 1:
            raise ValueError("Модуль должен быть больше 1")
        self.N = N
        self.modulus = modulus
        self.M = N + 1 # число мономов
        self.coeffs = [0] * self.M

        if coeffs is None:
            return

        if isinstance(coeffs, (list, tuple)):
            for i, val in enumerate(coeffs):
                if i < self.M:
```



```

        self.coeffs[i] = val % self.modulus

    elif isinstance(coeffs, dict):
        for deg, val in coeffs.items():
            if 0 <= deg < self.M:
                self.coeffs[deg] = val % self.modulus

    else:
        raise TypeError("coeffs должен быть списком, кортежем
или словарём")

    def __mod__(self, value):
        """Приводит целое число к остатку по модулю self.modulus"""
        return value % self.modulus

    def __getitem__(self, idx):
        return self.coeffs[idx]

    def __setitem__(self, idx, value):
        self.coeffs[idx] = self.__mod__(value)

    def __len__(self):
        return self.M

    def __add__(self, other):
        if not isinstance(other, CyclicPolynomial):
            raise TypeError("Можно складывать только с
CyclicPolynomial")

        if self.N != other.N or self.modulus != other.modulus:
            raise ValueError("Степени N и модули должны совпадать")

        res = CyclicPolynomial(self.N, self.modulus)
        for i in range(self.M):
            res[i] = self[i] + other[i]

        return res

    def __sub__(self, other):
        if not isinstance(other, CyclicPolynomial):
            raise TypeError("Можно вычитать только
CyclicPolynomial")

        if self.N != other.N or self.modulus != other.modulus:

```

```

        raise ValueError("Степени N и модули должны совпадать")

    res = CyclicPolynomial(self.N, self.modulus)
    for i in range(self.M):
        res[i] = self[i] - other[i]

    return res

def _mul_poly(self, other):
    """Умножение многочленов"""
    res = CyclicPolynomial(self.N, self.modulus)
    for i in range(self.M):
        for j in range(self.M):
            k = (i + j) % self.M
            res[k] += self[i] * other[j]

    return res

def __mul__(self, other):
    if isinstance(other, int):
        scalar = other % self.modulus
        res = CyclicPolynomial(self.N, self.modulus)
        for i in range(self.M):
            res[i] = self[i] * scalar
        return res

    if not isinstance(other, CyclicPolynomial):
        return NotImplemented

    if self.N != other.N or self.modulus != other.modulus:
        raise ValueError("Степени N и модули должны совпадать")

    return self._mul_poly(other)

def __rmul__(self, scalar):
    # scalar * self
    return self.__mul__(scalar)

def __pow__(self, exponent):
    if exponent < 0:
        raise ValueError("Степень должна быть неотрицательной")

    result = CyclicPolynomial(self.N, self.modulus, {0: 1}) #
x^0 = 1

```

```

    base = self
    while exponent:
        if exponent & 1:
            result = result * base
        base = base * base
        exponent >>= 1

    return result

def __eq__(self, other):
    if not isinstance(other, CyclicPolynomial):
        return False

    return (
        self.N == other.N
        and self.modulus == other.modulus
        and self.coeffs == other.coeffs
    )

def __str__(self):
    terms = []
    for i, c in enumerate(self.coeffs):
        if c == 0:
            continue
        if i == 0:
            terms.append(f"{c}")
        elif i == 1:
            if c == 1:
                terms.append("x")
            else:
                terms.append(f"{c}*x")
        else:
            if c == 1:
                terms.append(f"x^{i}")
            else:
                terms.append(f"{c}*x^{i}")

    if not terms:
        return "0"

    return " + ".join(terms)

def __repr__(self):
    return f"CyclicPolynomial(N={self.N},

```

```
modulus={self.modulus}), coeffs={self.coeffs}"
```

```
def degree(self):
    """Степень многочлена"""
    for i in range(self.M - 1, -1, -1):
        if self.coeffs[i] != 0:
            return i
    return -1
```

Модуль utils.py

```
import numpy as np
from CyclicPolynomial import CyclicPolynomial

def get_identity_poly_matrix(size, N, modulus):
    """Создает единичную матрицу из объектов CyclicPolynomial"""
    identity = np.empty((size, size), dtype=object)
    for i in range(size):
        for j in range(size):
            if i == j:
                identity[i, j] = CyclicPolynomial(N, modulus, {0:
1}) # Полином '1'
            else:
                identity[i, j] = CyclicPolynomial(N, modulus, {})
# Полином '0'
    return identity

def mat_pow(mat, exponent):
    """Возведение матрицы (np.ndarray, dtype=object) в целую
неотрицательную степень"""
    size = mat.shape[0]

    # Достаем параметры кольца из первого элемента матрицы
    sample_poly = mat[0, 0]
    N = sample_poly.N
    modulus = sample_poly.modulus

    result = get_identity_poly_matrix(size, N, modulus)
    base = mat
    e = exponent
    while e > 0:
```

```

        if e & 1:
            result = result @ base
        base = base @ base
        e >>= 1
    return result

def is_identity(mat):
    """Проверка, является ли матрица единичной"""
    size = mat.shape[0]

    sample_poly = mat[0, 0]
    N = sample_poly.N
    modulus = sample_poly.modulus

    identity = get_identity_poly_matrix(size, N, modulus)
    return np.array_equal(mat, identity)

```

Модуль protocol.py

```

import numpy as np
import random
from CyclicPolynomial import CyclicPolynomial
from utils import is_identity, mat_pow

def random_polynomial(N, modulus, density=0.5):
    """
    Генерирует случайный многочлен из CyclicPolynomial.
    density: вероятность того, что коэффициент при каждой степени
    ненулевой.
    """
    coeffs = {}
    for deg in range(N + 1):
        if random.random() < density:
            coeffs[deg] = random.randint(0, modulus - 1)
    return CyclicPolynomial(N, modulus, coeffs)

def generate_random_matrix(size, N, modulus, density=0.5):
    """
    Генерирует квадратную матрицу случайных многочленов.
    """

```

```

mat = np.empty((size, size), dtype=object)
for i in range(size):
    for j in range(size):
        mat[i, j] = random_polynomial(N, modulus, density)
return mat

def is_identity_matrix(mat):
    return is_identity(mat)

class StickelProtocol:
    """
    Реализация протокола Стикеля для матриц над кольцом циклических
    многочленов.
    Публичные параметры: матрицы A и B, такие, что  $AB \neq BA$ .
    """

    def __init__(self, A, B):
        """
        A, B: numpy массивы (dtype = object) размера d x d.
        """
        if A.shape != B.shape or A.ndim != 2 or A.shape[0] !=
A.shape[1]:
            raise ValueError(
                "A и B должны быть квадратными матрицами
одинакового размера"
            )

        self.A = A
        self.B = B
        self.size = A.shape[0]
        self.upper_bound = 10 * 6

    def alice_step(self):
        """Алиса: выбирает  $n < \text{upper\_bound}$ ,  $m < \text{upper\_bound}$  и
        вычисляет  $u = A^n * B^m$ ."""
        n = random.randint(0, self.upper_bound - 1)
        m = random.randint(0, self.upper_bound - 1)
        An = mat_pow(self.A, n)
        Bm = mat_pow(self.B, m)
        u = An @ Bm
        return u, n, m

```

```

def bob_step(self):
    """Боб: выбирает  $r < \text{orderA}$ ,  $s < \text{orderB}$  и вычисляет  $v = A^r$ 
    *  $B^s$ ."""
    r = random.randint(0, self.upper_bound - 1)
    s = random.randint(0, self.upper_bound - 1)
    Ar = mat_pow(self.A, r)
    Bs = mat_pow(self.B, s)
    v = Ar @ Bs
    return v, r, s

def alice_key(self, v, n, m):
    """Алиса вычисляет общий ключ:  $K_A = A^n * v * B^m$ """
    An = mat_pow(self.A, n)
    Bm = mat_pow(self.B, m)
    return An @ v @ Bm

def bob_key(self, u, r, s):
    """Боб вычисляет общий ключ:  $K_B = A^r * u * B^s$ """
    Ar = mat_pow(self.A, r)
    Bs = mat_pow(self.B, s)
    return Ar @ u @ Bs

def run(self):
    """Запускает полный протокол и возвращает ключи, а также
    промежуточные значения."""
    u, n, m = self.alice_step()
    v, r, s = self.bob_step()
    K_alice = self.alice_key(v, n, m)
    K_bob = self.bob_key(u, r, s)
    return {
        "u": u,
        "v": v,
        "n": n,
        "m": m,
        "r": r,
        "s": s,
        "K_alice": K_alice,
        "K_bob": K_bob,
        "keys_match": np.array_equal(K_alice, K_bob),
    }

```

Модуль app.py

```

import streamlit as st
import numpy as np

# Импортируем ваши классы
from CyclicPolynomial import CyclicPolynomial
from protocol import StickelProtocol, generate_random_matrix

def poly_to_latex(poly):
    """Преобразует CyclicPolynomial в строку LaTeX"""
    if not isinstance(poly, CyclicPolynomial):
        return str(poly)

    terms = []
    for i, c in enumerate(poly.coeffs):
        if c == 0:
            continue
        if i == 0:
            terms.append(f"{c}")
        elif i == 1:
            terms.append("x" if c == 1 else f"{c}x")
        else:
            terms.append(f"x^{{{i}}}" if c == 1 else f"{c}
x^{{{i}}}")

    if not terms:
        return "0"
    return " + ".join(terms)

def matrix_to_latex(mat):
    """Преобразует numpy матрицу полиномов в строку LaTeX"""
    rows, cols = mat.shape
    latex_str = r"\begin{pmatrix}" + "\n"
    for i in range(rows):
        row_strs = [poly_to_latex(mat[i, j]) for j in range(cols)]
        latex_str += " & ".join(row_strs) + r" \\" + "\n"
    latex_str += r"\end{pmatrix}"
    return latex_str

# --- ИНТЕРФЕЙС STREAMLIT ---
st.set_page_config(page_title="Stickel Protocol", layout="wide")
st.title("Протокол Стиклея над циклическими многочленами")

```



```

st.sidebar.header("Параметры генерации")
modulus = st.sidebar.number_input(
    "Модуль кольца ($n$)", min_value=2, max_value=100, value=7,
    step=1
)
N = st.sidebar.number_input(
    "Макс. степень многочлена ($N$)", min_value=1, max_value=10,
    value=3, step=1
)
matrix_size = st.sidebar.slider("Размер матриц", min_value=2,
max_value=4, value=2)
density = st.sidebar.slider(
    "Плотность (вероятность ненулевого коэффициента)",
    min_value=0.1,
    max_value=1.0,
    value=0.6,
)

if st.sidebar.button("Сгенерировать и запустить протокол",
type="primary"):
    # Генерация матриц, проверяем чтобы они не коммутировали (AB !=
BA)
    with st.spinner("Генерация матриц и выполнение протокола..."):
        while True:
            A = generate_random_matrix(matrix_size, N, modulus,
density)
            B = generate_random_matrix(matrix_size, N, modulus,
density)
            if not np.array_equal(A @ B, B @ A):
                break

        # Запуск протокола
        protocol = StickelProtocol(A, B)
        results = protocol.run()

# Вывод публичных матриц
st.header("1. Публичные параметры")
st.write(
    "Матрицы $A$ и $B$ над $\mathbb{Z}_{\text{"}}${"
    + str(modulus)
    + "}[x]/(x^{{"
    + str(N + 1)
    + "}}-1)$:"

```

```

)

col1, col2 = st.columns(2)
with col1:
    st.subheader("Матрица $A$")
    st.latex(r"A = " + matrix_to_latex(A))
with col2:
    st.subheader("Матрица $B$")
    st.latex(r"B = " + matrix_to_latex(B))

st.divider()

# Обмен ключами
st.header("2. Обмен сообщениями")

col_alice, col_bob = st.columns(2)
with col_alice:
    st.subheader("👩 Алиса")
    st.write(
        f"Генерирует секретные числа: $n = \{results['n']\}$, $m$
= $\{results['m']\}$"
    )
    st.write("Вычисляет и отправляет $u = A^n B^m$:")
    st.latex(r"u = " + matrix_to_latex(results["u"]))

with col_bob:
    st.subheader("👨 Боб")
    st.write(
        f"Генерирует секретные числа: $r = \{results['r']\}$, $s$
= $\{results['s']\}$"
    )
    st.write("Вычисляет и отправляет $v = A^r B^s$:")
    st.latex(r"v = " + matrix_to_latex(results["v"]))

st.divider()

# Вычисление общего ключа
st.header("3. Вычисление общего ключа")
st.write(
    "Каждая сторона вычисляет общий секрет: $K_A = A^n v B^m$ и
$K_B = A^r u B^s$."
)

col_k1, col_k2 = st.columns(2)

```

```

with col_k1:
    st.markdown("**Ключ, вычисленный Алисой ( $K_A$ ):**")
    st.latex(r"K_A = " + matrix_to_latex(results["K_alice"]))
with col_k2:
    st.markdown("**Ключ, вычисленный Бобом ( $K_B$ ):**")
    st.latex(r"K_B = " + matrix_to_latex(results["K_bob"]))

if results["keys_match"]:
    st.success("✅ Протокол успешно завершен! Ключи Алисы и Боба совпадают.")
else:
    st.error("❌ Ошибка! Ключи не совпадают.")

```